

Introduction to Processes and Inter-Process Communication

Mohammad Maksood Akhter, PhD
Assistant Professor

Department of Computer Science and Engineering

November 3, 2025



This lecture introduces the concept of **Processes** and how they interact using **Inter-Process Communication (IPC)**.

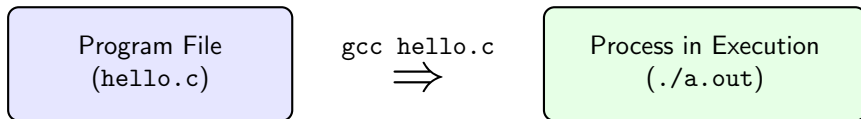
- Difference between Program and Process
- Process Memory Layout
- Process Creation using `fork()`
- Process Hierarchy (Tree)
- Introduction to IPC Mechanisms

Goal

Understand how a C program becomes a process, how processes are managed, and how they communicate with each other.

Concept

A **Program** is a passive entity stored on disk, while a **Process** is an active entity representing a running instance of that program.



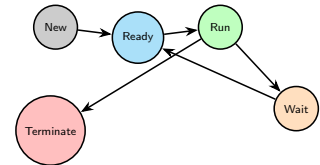
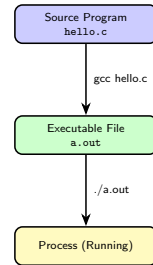
A **program** becomes a **process** after compilation and execution. The compiler (gcc) converts source code into an executable file, which the OS loads as an active process.

A **program** is a passive entity (set of instructions stored on disk), whereas a **process** is an active entity — a program in execution. The Operating System is responsible for creating and managing processes.

Lifecycle Overview:

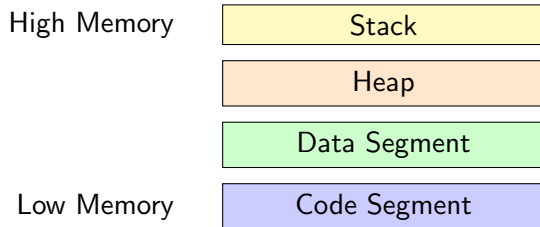
- Source code (`hello.c`) is compiled → creates an **Executable file** (`a.out`).
- When executed, the OS loads it into memory → becomes a **Process**.
- The process then passes through various **states**: New, Ready, Running, Waiting, and Terminated.
- Each process is uniquely identified by its **PID (Process ID)** and tracked via a **PCB (Process Control Block)**.

Thus, execution transforms static code into a dynamic entity managed by the OS.



Every process has its own separate memory divided into different regions:

- **Code Segment** – contains executable instructions.
- **Data Segment** – global and static variables.
- **Heap** – dynamically allocated memory.
- **Stack** – function calls and local variables.



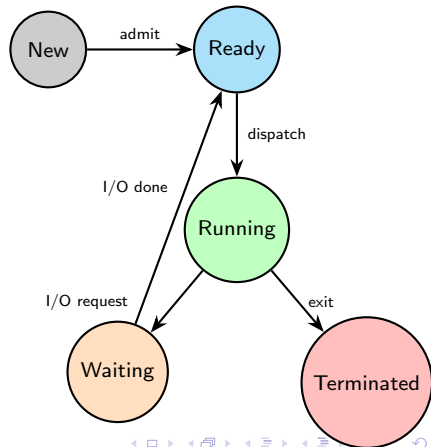
Each process passes through a series of states during its lifetime. These states describe what the process is currently doing.

- **New** – Process is being created.
- **Ready** – Loaded into memory, waiting for CPU.
- **Running** – Currently executing on CPU.
- **Waiting (Blocked)** – Waiting for an event (e.g., I/O).
- **Terminated** – Execution finished or aborted.

Transitions:

- New → Ready → Running → Terminated
- Running → Waiting → Ready (for I/O events)

The OS maintains each process's state information in the Process Control Block (PCB).



Every process in an operating system has a unique **Process ID (PID)** assigned by the OS. The process that starts another process is known as its **Parent Process**.

Example: Display Process ID and Parent Process ID

```
1  #include <stdio.h>
2  #include <unistd.h>
3  int main() {
4      printf("Process ID (PID): %d\n", getpid());
5      printf("Parent Process ID (PPID): %d\n", getppid());
6      return 0;
7  }
```

Explanation:

- `getpid()` → returns the unique ID of the running process.
- `getppid()` → returns the ID of the parent process (usually the shell).
- When you run this program, the OS creates a new process and assigns it a PID.

Objective: Observe that every running program is a process.

```
1      #include <stdio.h>
2
3      int main() {
4          printf("Hello , Operating System!\n");
5          while(1); // keep running to observe in process list
6          return 0;
7      }
```

Run and Observe:

- Compile → `gcc hello.c -o hello`
- Run → `./hello`
- In another terminal → `ps` or `top`
- You will see your program as a running **process**.

Takeaway: A `.c` program, when compiled and executed, becomes a **process** in memory.

Objective: Identify your program's process details.

```
1  #include <stdio.h>
2  #include <unistd.h>
3  int main() {
4      printf("Current Process ID (PID): %d\n", getpid());
5      printf("Parent Process ID (PPID): %d\n", getppid());
6      return 0;
7  }
```

Run:

- Compile → `gcc pid.c -o pid`
- Run → `./pid`

Sample Output:

```
Current Process ID (PID): 13247
Parent Process ID (PPID): 12210
```

Explanation:

- Each running process has a unique **PID**.
- Every process (except `init/systemd`) is created by another — the **parent process**.

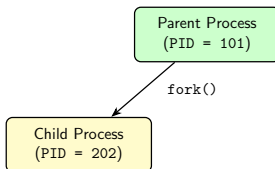
- 1 Write a C program to display the **Process ID (PID)** of the currently running process.
- 2 Modify the program to also display the **Parent Process ID (PPID)**.
- 3 Run your program multiple times — observe and record the **changes in PID**. Why does it change?

Objective: Understand how the OS assigns and manages process identifiers (PID and PPID).

Concept

In UNIX-like operating systems, a process can create a new process by calling the `fork()` system call.

- The calling process becomes the **parent**, and the newly created one is called the **child**.
- The `fork()` function duplicates the parent's memory image — the child receives an exact copy of the parent's address space.
- After a successful fork, both processes continue execution from the instruction **immediately following** the `fork()` call.
- Each process has its own **unique Process ID (PID)** and executes independently.



fork() Prototype:

- `#include <unistd.h>`
- `pid_t fork(void);`

```
1  #include <stdio.h>
2  #include <unistd.h>
3  int main() {
4      pid_t pid = fork(); // create a new process
5
6      if (pid == 0)
7          printf("Child Process\n");
8      else
9          printf("Parent Process\n");
10     return 0;
11 }
```

After a `fork()`, both parent and child run the same program code but have completely separate memory spaces. The return value of `fork()` distinguishes them:

- `pid == 0` → Child process
- `pid > 0` → Parent process (receives child's PID)
- `pid < 0` → Fork failed

Failure Condition:

- If the system lacks the necessary resources to create a child process or process limits would be exceeded, `fork()` sets `errno = EAGAIN`.
- In such cases, `fork()` does **not create a child process**.

When fork() Fails:

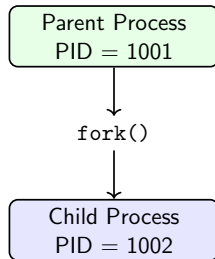
- fork() returns -1.
- The global variable errno is set to describe the failure reason.

Common Failure Reason:

- errno = EAGAIN — system cannot create a new process.
 - System reached its maximum process limit.
 - User exceeded per-user process quota.
 - Insufficient system resources (e.g., memory).
- In this case, fork() does **not create a child process**.

Example Code:

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <errno.h>
4
5  int main() {
6      pid_t pid = fork();
7
8      if (pid == -1) {
9          if (errno == EAGAIN)
10             printf("System limit reached: cannot
11                  create new process.\n");
12             else
13                 perror("fork failed");
14         }
15         else if (pid == 0)
16             printf("Child process running...\n");
17         else
18             printf("Parent process running...\n");
19
20         return 0;
21     }
```



When `fork()` is called:

- A new child process is created with a unique PID.
- The child inherits the parent's code, environment, and open files.
- Both processes continue execution independently after the `fork()` call.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     int x;
6     x = 0;
7     fork();
8     x = 1;
9     printf("I am process %ld and my x is %d\n",
10         (long)getpid(), x);
11     return 0;
12 }
```

Find the Output:

I am process 13247 and my x is 1
I am process 13248 and my x is 1

Explanation:

- fork() creates a new child process.
- Both parent and child continue execution after fork().
- Each has its own copy of variable x.
- Hence both print x = 1.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main(void) {
6     pid_t childpid;
7     childpid = fork();
8
9     if (childpid == -1) {
10         perror("Failed to fork");
11         return 1;
12     }
13
14     if (childpid == 0) /* child code */
15         printf("I am child %ld\n", (long)getpid());
16     else /* parent code */
17         printf("I am parent %ld\n", (long)getpid());
18     return 0;
19 }
```

Find the Output:

I am parent 13247

I am child 13248

Explanation:

- fork() returns **0** to child.
- fork() returns **child PID** to parent.
- Each branch executes independently.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main(void) {
6     pid_t childpid;
7     pid_t mypid;
8     mypid = getpid();
9     childpid = fork();
10
11     if (childpid == -1) {
12         perror("Failed to fork");
13         return 1;
14     }
15
16     if (childpid == 0) /* child code */
17         printf("I am child %ld, ID = %ld\n",
18             (long)getpid(), (long)mypid);
19     else /* parent code */
20         printf("I am parent %ld, ID = %ld\n",
21             (long)getpid(), (long)mypid);
22     return 0;
23 }
```

Find the Output:

I am parent 13247, ID = 13247

I am child 13248, ID = 13247

Explanation:

- mypid is captured before the fork.
- Child inherits the parent's variables, so its copy of mypid = parent's PID.
- But its own getpid() is unique.

Code:

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main(void) {
5      printf("A\t");
6      fork();
7      printf("B\t");
8      fork();
9      printf("C\t");
10     return 0;
11 }
```

Possible Output (Order may vary):

A B C B C C C C

Explanation:

- Two fork() calls $\rightarrow 2^2 = 4$ processes.
- Each process executes remaining statements after fork.
- Output order varies because processes run concurrently.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     pid_t pid = fork();
6     if (pid == 0) {
7         fork();
8         printf("Child section\n");
9     } else {
10        printf("Parent section\n");
11    }
12    return 0;
13 }
```

Output (order may differ):

Parent section

Child section

Child section

Explanation:

- Parent prints once.
- Child creates another child, so two “Child section” lines appear.
- Total 3 processes at the end.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     pid_t pid = fork();
6     if (pid == 0) {
7         fork();
8         printf("Child section\n");
9     } else {
10        printf("Parent section\n");
11    }
12    return 0;
13 }
```

Output (order may differ):

Parent section

Child section

Child section

Explanation:

- Parent prints once.
- Child creates another child, so two “Child section” lines appear.
- Total 3 processes at the end.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     pid_t pid = fork();
6     if (pid == 0)
7         printf("Child: PID=%ld, PPID=%ld\n",
8             (long)getpid(), (long)getppid());
9     else
10        printf("Parent: PID=%ld, PPID=%ld\n",
11            (long)getpid(), (long)getppid());
12    return 0;
13 }
```

Sample Output:

Parent: PID=1020, PPID=980

Child: PID=1021, PPID=1020

Explanation:

- getpid() → returns process's own ID.
- getppid() → returns parent's ID.
- Demonstrates parent-child linkage.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     int x = 5;
6     if (fork() == 0)
7         x += 5; // child modifies its own copy
8     else
9         x -= 5; // parent modifies its own copy
10
11     printf("PID %ld: x = %d\n",
12           (long)getpid(), x);
13     return 0;
14 }
```

Sample Output:

PID 13247: x = 0

PID 13248: x = 10

Explanation:

- Each process gets its own copy of variables.
- Changes in one process do not affect the other.

Code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     fork();
6     if (fork())
7         fork();
8     printf("Hello from %ld\n", (long)getpid());
9     return 0;
10 }
```

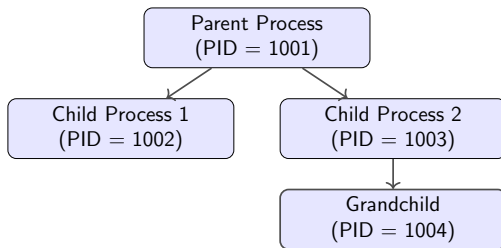
Expected Output (4 lines total):

```
Hello from 1301
Hello from 1302
Hello from 1303
Hello from 1304
```

Explanation:

- Three fork calls → 4 processes in total.
- Each prints "Hello" independently.

When multiple processes are created, they form a hierarchical structure known as a **process tree**. Each process can create one or more children, and all processes ultimately descend from the `init/systemd` process (PID = 1).



The `init/systemd` process is the root of all processes. Every new process is created by a parent, forming this hierarchical structure.

Concept

By using multiple `fork()` calls, a program can create several processes forming a hierarchical **process tree**. Each new process is an independent copy of the parent.

Demonstration: Process Tree using Multiple fork() Calls

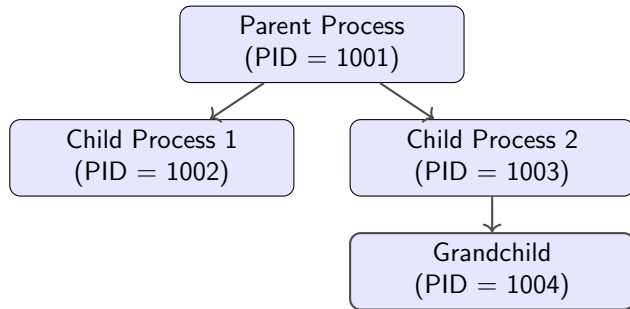


```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main() {
4     pid_t pid1, pid2;
5     pid1 = fork(); // first child
6     if (pid1 == 0) { // Child 1
7         printf("Child Process 1 (PID=%d, PPID=%d)\n", getpid(), getppid());
8     } else {
9         pid2 = fork(); // second child
10        if (pid2 == 0) { // Child 2
11            printf("Child Process 2 (PID=%d, PPID=%d)\n", getpid(), getppid());
12            pid_t pid3 = fork(); // grandchild
13            if (pid3 == 0) {
14                printf("Grandchild (PID=%d, PPID=%d)\n", getpid(), getppid());
15            }
16        } else {
17            printf("Parent Process (PID=%d)\n", getpid()); // Parent
18        }
19    }
20    return 0;}
```

Explanation:

- The first `fork()` creates **Child Process 1 (PID = 1002)**.
- The second `fork()` creates **Child Process 2 (PID = 1003)**.
- Inside Child 2, another `fork()` creates the **Grandchild (PID = 1004)**.
- The original **Parent Process (PID = 1001)** creates both children.

All these together form a process tree identical to the figure shown below.



Thank You!

Questions?